

Predavanje 12

Predikatska logika drugog reda

Izrazi predikatske logike prvog reda imaju veoma veliku izražajnu moć, *ali ne i dovoljnu*. Recimo, postoji veliki broj veoma značajnih i naoko jednostavnih matematičkih tvrdjenja koja se ne mogu iskazati izrazima predikatske logike prvog reda. Pored toga, pokazano je da se uz pomoć predikatske logike prvog reda mogu formalno opisati samo oni problemi za koje postoji algoritam čije je vrijeme izvršavanja *polinomska funkcija* od veličine problema, a mnogi problemi kompjuterskih nauka *nisu takvi*. Stoga su prirodno vršeni pokušaji da se izražajna moć predikatske logike prvog reda poveća. Jedan od takvih pokušaja je i **predikatska logika drugog reda**.

Da bismo uvidjeli ograničenu moć izražavanja predikatske logike prvog reda, uzmimo kao primjer *princip matematičke indukcije*, koji opisno možemo iskazati kao "svako svojstvo koje vrijedi za nulu i za koje iz pretpostavke da ono vrijedi za neki proizvoljan prirodan broj n slijedi da ono vrijedi i za njegovog sljedbenika n' , vrijedi za sve prirodne brojeve". Uzmemo li skup prirodnih brojeva za domen interpretacije, razuman pokušaj da zapišemo ovaj princip korištenjem jezika predikatske logike glasi

$$\forall P (P(0) \wedge \forall n (P(n) \Rightarrow P(n')) \Rightarrow \forall n P(n))$$

Ovdje smo sa P označili svojstvo o kojem se govori. Ovo zaista jeste korektan zapis principa matematičke indukcije, ali ovo *nije izraz predikatske logike prvog reda*. Zaista, u predikatskoj logici prvog reda kvantifikatori mogu djelovati *samo na promjenljive*, a u ovom izrazu smo kvantifikator " $\forall$ " primijenili na *predikat* (u pokušaju da iskažemo frazu "svako svojstvo"). Detaljnija razmatranja ukazuju na to da princip matematičke indukcije uopće *nije moguće* iskazati isključivo korištenjem predikatske logike prvog reda.

Ilustrirajmo potrebu za generalizacijom predikatske logike prvog reda još jednim primjerom. Neka želimo konstruisati formulu predikatske logike koja je tačna ako i samo ako je domen interpretacije *beskonačan skup* (takva formula će zapravo iskazivati *tvrdnju da je domen beskonačan skup*). Podsjetimo se da je, po Dedekindovoj definiciji beskonačnog skupa, skup beskonačan ukoliko postoji bijekcija između njega i nekog njegovog pravog podskupa. Probajmo jezikom predikatske logike iskazati egzistenciju takve bijekcije:

$$\exists f (\forall x \forall y (x \neq y \Rightarrow f(x) \neq f(y)) \wedge \exists z \forall x (f(x) \neq z))$$

Ovaj zapis je također korektan sa logičkog aspekta, ali se ponovo ne radi o izrazu predikatske logike prvog reda, s obzirom da se u njemu kvantifikator " $\exists$ " primjenjuje na *funkciju*, a ne na *promjenljivu*. I u ovom primjeru je moguće dokazati da jezikom predikatske logike prvog reda uopće nije moguće iskazati tvrdnju o beskonačnosti domena (niti o konačnosti domena).

Prethodna dva primjera ukazuju da bi se izražajnost predikatske logike mogla povećati ukoliko bismo dozvolili da kvantifikatori pored promjenljivih mogu djelovati i na *funkcije* i *predikate*. To je upravo učinjeno u *predikatskoj logici drugog reda*. Preciznije, predikatska logika drugog reda, pored klasičnih promjenljivih, funkcijskih simbola i predikatskih simbola, uvodi *funkcijske promjenljive* i *predikatske promjenljive*. Za razliku od funkcijskih i predikatskih simbola, koji uvijek predstavljaju neku *konkretnu funkciju* odnosno neki *konkretni predikat* (zavisno od izabrane interpretacije), funkcijske i predikatske promjenljive predstavljaju *proizvoljnu funkciju* (definiranu nad domenom interpretacije) odnosno *proizvoljan predikat* (definiran nad domenom interpretacije). Pri tome se na funkcijske i predikatske promjenljive također mogu primjenjivati kvantifikatori. Recimo, u prethodna dva primjera, P odnosno f su predstavljali predikatsku odnosno funkcijsku promjenljivu respektivno.

U mnogim definicijama predikatske logike drugog reda se za kvantifikatore dopušta da, umjesto na predikate, djeluju na *skupovne promjenljive*, za koje se predstavlja da mogu predstavljati razne podskupove domena interpretacije (tj. njima se formaliziraju konstrukcije poput "Za svaki podskup domena vrijedi ..." odnosno "Postoji podskup domena za koji vrijedi ..."). Međutim, takva definicija je

ekvivalentna sa definicijom u kojoj kvantifikatori djeluju na predikate, s obzirom da svaki predikat definira neki podskup domena i, obrnuto, svaki podskup domena može se modelirati nekim predikatom koji je tačan samo za one elemente domena koji pripadaju tom podskupu.

Ponekad se u predikatskoj logici drugog reda, radi smanjenja opasnosti od nesporazuma, funkcijske odnosno predikatske promjenljive označavaju ponešto drugačijim oznakama u odnosu na klasične funkcijske odnosno predikatske simbole. Tako se funkcijske promjenljive često označavaju malim slovima poput u, v, w itd. (za razliku od slova f, g, h itd. koja su tipično rezervirana za obične funkcijske simbole), ili malim grčkim slovima poput φ, ψ itd. dok se predikatske promjenljive često označavaju velikim slovima poput X, Y, Z itd. (za razliku od slova P, Q, R itd. koja su tipično rezervirana za obične predikate odnosno malih slova x, y, z itd. koja su tipično rezervirana za klasične promjenljive), ili velikim grčkim slovima poput Π, Σ, Γ itd. S obzirom da ćemo se sa predikatskom logikom drugog reda upoznati samo informativno, ovdje nećemo insistirati na razdvajanju oznaka za funkcije i funkcijske promjenljive, odnosno predikate i predikatske promjenljive.

Pokažimo na još nekoliko primjera izražajnu razliku između predikatske logike prvog i drugog reda. Ukoliko želimo u logici prvog reda iskazati da je neka funkcija f identička funkcija, to možemo lako zapisati kao $\forall x (f(x) = x)$. Međutim, želimo li iskazati da *postoji identička funkcija*, ne možemo koristiti logiku prvog reda, već moramo koristiti izraz predikatske logike drugog reda oblika $\exists f \forall x (f(x) = x)$. Dalje, u predikatskoj logici prvog reda ne možemo iskazati pretpostavku koja tvrdi da svaka dva objekta (iz nekog skupa) posjeduju barem neko zajedničko svojstvo, dok takvu pretpostavku lako iskazujemo u predikatskoj logici drugog reda izrazom poput $\forall x \forall y \exists P (P(x) \wedge P(y))$. Još jedan karakterističan primjer je poznata **Leibnizova definicija jednakosti** po kojoj su dva objekta x i y jednaki ako i samo ako svako svojstvo koje posjeduje objekat x posjeduje i objekat y (tako da *nema načina* na koji bi ih mogli razlikovati). Ovu definiciju je nemoguće iskazati jezikom predikatske logike prvog reda, ali ju je posve lako iskazati jezikom predikatske logike drugog reda u vidu izraza $(x = y) \Leftrightarrow \forall P (P(x) \Leftrightarrow P(y))$. U logici prvog reda sve što možemo reći o tome je to da ukoliko su dva objekta jednaka, tada *za neko konkretno svojstvo* P ili ga oba posjeduju, ili ga oba ne posjeduju, odnosno izreći izraz predikatske logike prvog reda poput $(x = y) \Rightarrow (P(x) \Leftrightarrow P(y))$.

Predikatska logika prvog reda nije u stanju čak ni da napravi razliku između *prebrojivih* i *neprebrojivih* (odnosno *diskretnih* i *nediskretnih*) skupova. Naime, za svaku zadovoljivu formulu predikatske logike prvog reda koja je tačna u nekoj interpretaciji sa neprebrojivim domenom, postoji i interpretacija sa prebrojivim domenom u kojoj je ona također tačna (ovo tvrđenje je poznato kao **Skolem-Löwenheimov teorem**). Stoga jezikom predikatske logike prvog reda nije moguće iskazati tvrdnju da je domen interpretacije prebrojiv odnosno da nije prebrojiv. Međutim, postoji mnogo načina da se to iskaže jezikom predikatske logike drugog reda. Jedan od načina da se to iskaže je sljedeći. Lako se uviđa da je u svakom prebrojivom skupu moguće iskazati neku verziju principa *matematičke indukcije*, tj. uvijek možemo naći neki element analogan elementu 0 u skupu prirodnih brojeva sa nulom (označimo taj element sa z) i neku funkciju analognu funkciji "sljedbenik od" u skupu prirodnih brojeva (označimo tu funkciju sa s), takvu da vrijedi princip analogan principu matematičke indukcije u skupu prirodnih brojeva. Iskažimo jezikom predikatske logike drugog reda tvrdnju da takav element z i takva funkcija s postoje:

$$\exists z \exists s \forall P (P(z) \wedge \forall n (P(n) \Rightarrow P(s(n))) \Rightarrow \forall n P(n))$$

S obzirom na gore provedeno razmatranje, ovaj izraz je tačan u svakoj interpretaciji čiji je domen prebrojiv skup. S druge strane, niti u jednom skupu koji nije prebrojiv nije moguće uspostaviti nikakvu direktnu analogiju principa matematičke indukcije, tako da navedeni izraz ne može biti tačan ni u kakvoj interpretaciji sa neprebrojivim domenom. Dakle, navedeni izraz je tačan u onim i samo onim interpretacijama sa prebrojivim domenom, te se može smatrati zapisom tvrdnje da je domen interpretacije prebrojiv skup.

Iako je predikatska logika drugog reda mnogo izražajnija od predikatske logike prvog reda, postoje ozbiljni problemi vezani za njenu primjenu. Na prvom mjestu, ispitivanje valjanosti, kontradiktornosti i zadovoljivosti izraza logike drugog reda je veoma težak problem, i u općem slučaju nije uopće rješiv. Moguće je dokazati da u logici drugog reda *ne postoji niti efektivan pozitivan test niti efektivan negativan*

test za ma koje od ova tri svojstva (valjanost, kontradiktornost i zadovoljivost). Recimo, pokazuje se da se za svaki izraz predikatske logike prvog reda može konstruisati izraz predikatske logike drugog reda koji je *valjan* ako i samo ako je polazni izraz predikatske logike prvog reda *zadovoljiv*. Stoga bi se svaki efektivan pozitivan test za *valjanost* izraza predikatske logike drugog reda mogao lako transformirati u efektivan pozitivan test za *zadovoljivost* izraza predikatske logike prvog reda, a već smo rekli da takav test ne postoji. Čak je i ispitivanje tačnosti jednog partikularnog izraza predikatske logike drugog reda pri jednoj partikularnoj interpretaciji često povezano sa brojnim teškoćama, pa čak i kontraverzama. Recimo, u predikatskoj logici drugog reda je moguće formirati izraz koji je tačan ako i samo ako vrijedi *hipoteza kontinuuma*, a već smo ranije naglasili koliko je to kontradiktorno pitanje (da bismo iskazali takvu tvrdnju, dovoljno je iskazati egzistenciju bijektivne funkcije čiji je domen neki poznati skup kardinalnog broja $\aleph_1$, a kodomen skup $\mathbb{R}$).

Jedan od razloga zbog čega za predikatsku logiku drugog reda nije moguće formirati test sličan onom koji je prezentiran za predikatsku logiku prvog reda leži u činjenici da za predikatsku logiku drugog reda ne važi *teorema o kompaktnosti* (odnosno *Herbrandova teorema*), tako da odbijenice, čak i kada postoje, ne moraju nužno imati *konačno mnogo članova*, a jasno je da je odbijenice sa beskonačno mnogo članova nemoguće generirati u konačnom vremenu. Interesantno je da kada bi postojao efektivan pozitivan test za testiranje valjanosti izraza predikatske logike drugog reda, taj test bi se lako mogao iskoristiti za pouzdano dokazivanje odnosno opovrgavanje tačnosti svih teorema aritmetike, s obzirom da se čitava aritmetika lako može opisati jezikom predikatske logike drugog reda (ali ne i prvog reda, zbog principa matematičke indukcije). S obzirom da je ranije dokazano da univerzalan mehanički postupak (algoritam) za dokazivanje odnosno opovrgavanje teorema aritmetike *ne postoji* (posljedica čuvene **Gödelove teoreme o nekompletnosti aritmetike**), to je još jedan dokaz činjenice da ne postoji efektivan pozitivan test za testiranje valjanosti izraza predikatske logike drugog reda.

Predikatska logika drugog reda je problematična i zbog toga što mnogi poznati i važni teoremi koje vrijede za predikatsku logiku prvog reda više *ne vrijede* u predikatskoj logici drugog reda. Na neki način, možemo reći da je predikatska logika drugog reda “previše izražajna”, u toj mjeri da to postaje pomalo “opasno”. Neki logičari, kao što je recimo **Quine**, tvrde da predikatska logika drugog reda “uopće nije logika”. Njegov argument je sljedeći. Logika bi trebala da bude apstraktna disciplina koja formalizira principe ispravnog zaključivanja, ne govoreći pri tome *ni o čemu određenom*. Posebno, logika se ne bi trebala oslanjati na postojanje formalno zasnovane teorije skupova (zapravo, vrijedi upravo obrnuto – predikatska logika prvog reda tipično se koristi *za formalno zasnivanje* same teorije skupova). Međutim, predikatska logika drugog reda, samim tim što dopušta primjenu kvantifikatora na funkcije i predikate (ili, alternativno, na podskupove domena), zahtijeva prethodno *zasnivanje* teorije skupova, da bi se preciziralo značenje pojmova poput “funkcija”, “predikat nad skupom” ili “podskup”. Slijedi da za predikatsku logiku drugog reda ne vrijedi da ona ne govori “ni o čemu određenom”, već o nečemu sasvim određenom: o *funkcijama* i *predikatima*, odnosno, općenitije, o *skupovima* (za razliku od logike prvog reda, koja skupove, funkcije i predikate koristi samo *kao pomoćno sredstvo*, logika drugog reda govori o *njima*). Stoga se ona ne uklapa u opći stav po kojem logika ne bi trebala govoriti i o kakvim određenim objektima, tako da logika drugog reda više liči na neku vrstu teorije skupova, nego na logiku u klasičnom smislu. Mada ovi argumenti nisu posve bez osnova, oni su, po mišljenjima mnogih, isuviše radikalni i nisu dovoljni da se predikatska logika drugog reda ne bi smatrala logikom.

Nekada se u logici drugog reda dopušta da funkcije imaju kao svoje argumente druge funkcije i predikate, kao i da predikati imaju kao svoje argumente druge funkcije i predikate. Recimo, sasvim je smisleno definirati predikate poput T i B koji redom znače “biti tranzitivan” i “biti bijektivan”, i koji, primijenjeni na neki drugi predikat odnosno funkciju, recimo u izrazima poput $T(P)$ ili $B(f)$, iskazuju da predikat P predstavlja neku tranzitivnu relaciju, odnosno da funkcija f predstavlja neku bijektivnu funkciju. Na taj način se još više pojačavaju izražajne mogućnosti logike, ali se isto tako lako stvaraju i nove nevolje. Na primjer, moguće je formirati predikat P koji je tačan kada mu ponudimo njega samog kao argument, odnosno za koji je $P(P) = T$. Na primjer, definirajmo da je neko svojstvo *uobičajeno* ukoliko ga posjeduje barem nekoliko objekata; tada je samo svojstvo “biti uobičajen” uobičajeno, jer postoji barem nekoliko uobičajenih svojstava, odnosno svojstvo “uobičajenosti” posjeduje samo sebe). Međutim, odavde se lako izvode paradoksi Russellovog tipa. Recimo, definirajmo da je neko svojstvo

obično ukoliko ono ne posjeduje samo sebe. Tada, slično kao kod Russellovog paradoksa, lako zaključujemo da je svojstvo obično ako i samo ako nije obično, što je tipičan paradoks.

Izlaz iz opisane paradoksalne situacije može se izvesti definiranjem **reda predikata**, tako da predikati prvog reda mogu imati samo obične objekte kao argumente, predikati drugog reda mogu imati kao argumente obične objekte i predikate prvog reda (ali ne i predikate drugog reda), itd. Ovo vodi ka kreiranju **logika višeg reda**. Logike višeg reda su nekada neophodne, ali uzimajući u obzir kakvi problemi postoje već kod logike drugog reda, nije teško naslutiti da se povećanjem reda logike drastično povećava i količina problema pri radu sa takvom logikom.

Elementi teorije izračunljivosti

Predmet proučavanja teorije izračunljivosti

Teorija izračunljivosti (engl. *theory of computability*) je relativno mlada grana matematičke logike čiji je cilj *ispitivanje do kojih granica mogu ići šablonizirani postupci koji se mogu mehanički provoditi nad nekom skupinom ulaznih podataka sa ciljem dobijanja rješenja*, odnosno koja klasa (vrsta) problema se može rješavati takvim “mehaniziranim” postupcima, *bez pozivanja na intuiciju, inspiraciju i inteligenciju*. Recimo, ranije smo tvrdili da ne postoji nikakav efektivno izvediv šablonski postupak koji bi za ma kakvu formulu predikatske logike prvog reda sa sigurnošću utvrdio da je valjana ili nije. Ova tvrdnja je upravo primjer tvrdnje izvedene primjenom teorije izračunljivosti. Podsjetimo se da postoje postupci koji u slučaju da formula jeste valjana to mogu garantirano utvrditi. Takvi postupci mogu i za mnoge formule koje nisu valjane utvrditi da nisu valjane, s obzirom da u nekom koraku pri izvođenju postupka prosto dolazimo u situaciju da smo iscrpili sve mogućnosti, a da valjanost formule nije potvrđena. Međutim, kod izvjesnih formula koje nisu valjane, lako možemo upasti u situaciju gdje se predloženi postupci nikada ne završavaju, jer se neprestano generiraju nove mogućnosti koje treba testirati. Neko bi mogao reći da to možda vrijedi za sve *do sada poznate postupke* za testiranje valjanosti, ali da to nije garancija da u budućnosti neće biti pronađen neki postupak koji prevazilazi ove probleme. Tu upravo stupa na scenu teorija izračunljivosti, koja tvrdi da efektivan mehanički postupak za utvrđivanje ili opovrgavanje valjanosti formula predikatskog računa prvog reda ne samo da nije dosad pronađen, već *nikad neće ni biti*, jer jednostavno *ne postoji*.

Za teoriju izračunljivosti se obično kaže da je veoma depresivna oblast matematike, s obzirom da glavnina njenih rezultata tvrdi da neki problemi koji su dugo privlačili pažnju matematičara zapravo u općem slučaju *nisu rješivi*. To ne znači da oni nisu rješivi u izvjesnim specijalnim slučajevima, ali svaki od tih specijalnih slučajeva *zahtijeva drugačiju strategiju rješavanja*, odnosno ne postoji *opći postupak* koji bi obuhvatio *sve slučajeve* (štaviše, mogu se naći i specijalni slučajevi koji nisu rješivi, niti će ikada biti). Ti rezultati su srušili snove mnogih matematičara koji su bili ubjeđeni da se *čitava matematika može formalizirati*. Recimo, **D. Hilbert** je čvrsto vjerovao da je moguće pronaći formalni postupak koji za ma kakvu teoriju za koju je poznat skup osnovnih tvrdjenja (aksioma) i pravila izvođenja koja propisuju kako se iz osnovnih tvrdjenja mogu izvoditi nova tvrdjenja ustanovljava da li neko zadano tvrdjenje slijedi ili ne slijedi iz polaznih aksioma u skladu sa datim pravilima izvođenja (odnosno utvđuje da li je neko zadano tvrdjenje *teorema* te teorije ili nije). Na žalost, teorija izračunljivosti je utvrdila da takav formalni postupak ne postoji. Još jedan poznati negativni rezultat teorije izračunljivosti vezan je za teoriju tzv. **Diofantovih jednačina**, što su zapravo polinomske jednačine sa više promjenljivih i cjelobrojnim koeficijentima na koje se nameću dodatni zahtjevi da i sva rješenja *moraju također biti cijeli brojevi* (npr. jednačine poput $2x + 3y + 5z = 100$ ili $x^2 - 5y^2 = x + 1$ uz dodatna ograničenja da x, y odnosno z budu cijeli brojevi). Na kongresu matematičara održanom 1900. godine Hilbert je prezentirao listu od 23 neriješena problema za koje je smatrao da bi nalaženje njihovog rješenja moglo biti od izuzetne koristi za matematiku (ti problemi su poznati kao **Hilbertovi problemi**). Među njima se posebno ističe **deseti Hilbertov problem** koji traži da se pronađe algoritam koji će za proizvoljnu Diofantovu jednačinu utvrditi da li je rješiva ili ne (npr. prethodne dvije jednačine jesu rješive, dok Diofantova jednačina $x^2 + y^2 = 6$ uz $x \in \mathbb{Z}$ i $y \in \mathbb{Z}$ nije). Primijetimo da Hilbert čak ne traži da se nađe algoritam za *rješavanje* takvih jednačina, nego samo da se utvrdi da li su rješive ili ne (što djeluje kao mnogo lakši problem). Isto tako, primijetimo da je Hilbert formirao problem u formi “Naći algoritam...” što znači da mu ni na kraj pameti nije bilo da bi se

moglo desiti da takav algoritam ne postoji (a upravo to će se pokazati kao istina). Za Diofantove jednačine prvog i drugog stepena (tj. linearne i kvadratne Diofantove jednačine) odavno je poznato ne samo kako utvrditi da li su rješive ili ne nego i kako naći njihovo rješenje, dok za Diofantove jednačine trećeg reda takav postupak u općem slučaju nije poznat (ali nije dokazano ni da ne postoji). Međutim, 1970. godine **Y. Matiyasevič** (nakon 20 godina rada) dokazuje jednu prividno nevezanu teoremu (poznatu kao **Matiyasevičeva teorema**) čija je direktna posljedica *nerješivost desetog Hilbertovog problema*. Tačnije, iz ove teoreme slijedi da ne postoji algoritam koji bi mogao testirati rješivost proizvoljne Diofantove jednačine čiji je stepen veći ili jednak od 4. O ovome ćemo kasnije detaljnije govoriti.

Kako se teorija izračunljivosti razvijala, otkrivano je sve više i više problema koji nisu algoritamski rješivi, što je dovelo do prilično depresivnog raspoloženja među matematičarima i osnovane sumnje u upotrebljivost računara za rješavanje ozbiljnih matematičkih problema. U teoriji izračunljivosti postoji čak i teorema, poznata kao **Riceova teorema** koja, veoma slobodno rečeno, tvrdi da praktično niti jedan problem koji se svodi na testiranje pripadnosti određenim podskupovima skupa prirodnih brojeva nije rješiv u svojoj najopštijoj formulaciji (a takvih problema je zaista veoma mnogo). Neki ovu teoremu citiraju kao "Ništa što je interesantno nije rješivo u najopćenitijoj formulaciji" (engl. "Nothing interesting is solvable in full generality").

Teorija izračunljivosti također je postavila ograničenja ne samo na mogućnost formalnog mehaničkog rješavanja izvjesnih klasa problema, već i na mogućnost samog formalnog *zasnivanja* izvjesnih oblasti matematike. Tako, na primjer, čuvena **Gödelova teorema o nekompletnosti** tvrdi da svaka neprotivrječna matematička teorija koja je dovoljno izražajna da se u njoj mogu zasnovati prirodni brojevi i operacije i relacije sa njima (tj. koja je dovoljno izražajna da opiše *zakone aritmetike*) mora biti *nepotpuna*, u smislu da je uvijek moguće formirati tačne tvrdnje koje se tiču pojmova te teorije, ali koje nisu dokazive unutar te teorije (tj. nisu izvodive iz njenih aksioma). Intuitivno rečeno, u svakoj dovoljno bogatoj matematičkoj teoriji, uvijek postoje "rupe" u smislu istinitih tvrdnji koje nisu dokazive unutar same teorije. Razumije se da, kad god formiramo neku takvu istinitu tvrdnju, možemo proširiti teoriju dodavanjem te tvrdnje kao aksioma. Međutim, i u takvoj proširenoj teoriji, ponovo je moguće formirati istinitu tvrdnju koja nije dokaziva u tako proširenoj teoriji, itd. Ovaj postupak se može produžiti unedogled i nikada nećemo doći do teorije u kojoj su sve istinite tvrdnje dokazive, odnosno u kojoj smo "zakrpali" sve rupe. Drugim riječima, postojanje "rupa" u teoriji je inherentno i neizbježno svojstvo svake dovoljno "bogate" matematičke teorije, koje se jednostavno *ne može izbjeći*. Jasno je da je ovakav zaključak u priličnoj mjeri uzdrmao same temelje matematike kao nauke, bar u onom njenom dijelu koji se oslanja na njene formalističke aspekte.

Teorija izračunljivosti u tijesnoj vezi je sa **teorijom algoritama** i predstavlja njen ekstremni dio koji zapravo ispituje *krajnje domete same teorije algoritama*. Mi kada tvrdimo da za neki problem ne postoji efektivni mehanički postupak za njegovo rješavanje, zapravo tvrdimo da ne postoji **algoritam** za rješavanje tog problema. Mada je već iz ove konstatacije intuitivno jasno šta algoritam predstavlja (odnosno, riječ "algoritam" koristimo kao puki sinonim za frazu "efektivni mehanički postupak"), očigledno je da za izricanje tako hrabre tvrdnje da za rješavanje nekog problema *ne postoji nikakav algoritam* moramo u potpunosti raščistiti s tim šta uopće podrazumijevamo pod tim pojmom. Začudo, ovaj relativno elementaran pojam izuzetno je teško definirati na način koji ne ostavlja nikakve dileme i nedoumice. U narednom odjeljku osvrnućemo se na neke klasične definicije pojma algoritma i konsekvence koje slijede iz takvih definicija.

Pojam algoritma

U brojnoj literaturi često se mogu susresti različite definicije algoritama, od kojih je većina u manjoj ili većoj mjeri nedovoljno precizna (ovo se naročito odnosi na srednjoškolsku literaturu). Definicije algoritama mogu se podijeliti na *neformalne* (intuitivno jasne, ali nematematičke) i *formalne* (iskazane strogim matematičkim jezikom). Jednu od najpreciznijih *neformalnih* definicija algoritma dao je **D. Knuth**, po kojem je algoritam *svaki jasan i nedvosmislen konačan niz koraka, koji su dovoljno prosti u smislu da se mogu izvoditi posve mehanički, tj. automatski bez posezanja za bilo kakvom inspiracijom, intuicijom ili inteligencijom i koji nakon konačno mnogo primijenjenih koraka na osnovu zadanih ulaznih podataka koji predstavljaju opis problema dovodi do rješenja problema ili odgovora da rješenja nema*. Inače, sama riječ

algoritam potiče od nepravilnog izgovaranja posljednje riječi u imenu poznatog uzbeko-persijskog matematičara iz devetog vijeka **Al-Khwarizmija**. On je u svom kapitalnom djelu iz 825. godine *Kitab al-jabr wa'l-muqabala* (primjetimo da iz naziva djela potiče riječ *algebra*) prvi formulisao jasne i precizne postupke (dakle, algoritme) za obavljanje četiri osnovne računske operacije pomoću papira i olovke.

Važno je napomenuti da su svi dijelovi prethodne definicije bitni. Recimo, u literaturi se često zaboravlja insistirati na *konačnosti* niza koraka koji čine algoritam i potrebe da algoritam dovodi do rješenja nakon *konačno mnogo primijenjenih koraka* (kažemo da postupak mora da **terminira** nakon konačno mnogo primijenjenih koraka). U suprotnom, ukoliko bismo imali postupak koji "vodi" ka rješavanju problema nakon primjene beskonačno mnogo koraka, onda je upitno možemo li uopće reći da taj postupak zaista vodi ka rješenju. Na primjer, trisekcija ugla, odnosno podjela ugla na tri jednaka dijela *samo primjenom linijara i šestara*, izvodljiva je u beskonačno mnogo, ali ne i u konačno mnogo koraka. Stoga se trisekcija ugla smatra nerješivim problemom, iako se u konačno mnogo koraka možemo približiti rješenju onoliko koliko god to želimo. Naravno, jasno je da se u ovom primjeru radi se samo o *teorijskoj* ali ne i o *praktičnoj* nerješivosti, s obzirom da za *praktične potrebe* možemo smatrati da smo trisekciju izvršili kada postignemo onoliko tačnost koja se zahtjeva u konkretnoj praktičnoj primjeni. Za probleme takve vrste kažemo da su **aproksimativno algoritamski rješivi** sa po volji zadanom tačnošću. Inače, ukoliko iz definicije algoritma izbacimo zahtjev da on mora terminirati nakon konačno mnogo koraka za sve ulazne podatke, dobijamo nešto što se naziva **algoritamski proces**. Jasno je da je svaki algoritam ujedno i algoritamski proces, ali obrnuto ne vrijedi.

Pomenuti problem konačnosti nije uopće tako naivan koliko na prvi pogled izgleda. Osnovni problem je u tome što je moguće formirati jasno i precizno definirane postupke (algoritamske procese) za koje nije nimalo jasno da li terminiraju nakon konačno mnogo primijenjenih koraka ili ne. Drugim riječima, nije jasno da li takvi postupci mogu biti osnova za formiranje algoritma ili ne. Klasični primjer je postupak koji su, neovisno jedan od drugog, formirali **S. Ulam** i **L. Collatz**. Oni su posmatrali postupak dat sljedećim nizom koraka (koji se obično naziva **Collatzov proces**):

1. *Zadati neki prirodan broj;*
2. *Ukoliko je broj paran, podijeliti ga sa 2, a u suprotnom, pomnožiti ga sa 3 i dodati 1 na rezultat;*
3. *Ukoliko je novodobijeni broj jednak 1, završiti postupak, a u suprotnom, vratiti se nazad na korak 2.*

Neka je, na primjer, u koraku 1. zadan prirodan broj 7. Primjenom predloženog postupka dobićemo niz brojeva 7, 22, 11, 34, 17, 52, 26, 13, 40, 20, 10, 5, 16, 8, 4, 2 i 1, nakon čega postupak terminira. Dakle, postupak sigurno terminira ukoliko se na početku kao ulazni podatak zada broj 7. Prirodno je postaviti pitanje da li ovaj postupak terminira ukoliko se kao ulazni podatak zada *ma kakav prirodan broj*. Koliko god ovo pitanje izgledalo naivno, odgovor na njega do danas *nije poznat* (mada je za odgovor raspisana nagrada od 100000 \$, zbog velike važnosti nalaženja odgovora na ovo pitanje u teorijskom računarstvu). Kako do sada stoje stvari, izgleda da je odgovor potvrđan (ovo vjerovanje poznato je kao **Collatzova hipoteza**). Naime, do danas su grubom silom (uz pomoć ogromne količine računarskih resursa) kao ulazni podaci testirani svi prirodni brojevi sa manje od 40 cifara i postupak je uvijek terminirao (ponekad nakon zaista astronomskog broja koraka). Drugim riječima, kontraprimjer za koji opisani postupak ne terminira do sada nije pronađen, ali to još nije dokaz da takav kontraprimjer ne postoji. Stoga je prirodno postaviti pitanje da li gore opisani postupak možemo smatrati algoritmom ili ne. S obzirom da se od algoritma zahtijeva njegovo terminiranje za sve ulazne podatke, dolazimo do interesantnog i neočekivanog odgovora da se *ne zna* da li prethodni postupak predstavlja algoritam ili ne.

Drugi važan aspekt u definiciji algoritma je zahtjev da se algoritam sastoji isključivo od *jasnog i nedvosmislenog niza koraka*, za čije izvođenje nije potrebna nikakva *inspiracija* niti *intuicija* (engl. *insight*). Činjenica je da rješavanje mnogih problema zahtijeva izvjesnu dovrtljivost i inspiraciju, tako da postupci kojim se dolazi do rješenja nisu nikako očigledni niti mehanizirani. Mada ni sintagma "jasan i nedvosmislen niz koraka" nije sama po sebi precizno (jasno i nedvosmisleno) definirana, očigledno je da kod takvih postupaka ne možemo govoriti o "jasnom i nedvosmislenom nizu koraka", odnosno da takve postupke ne možemo smatrati algoritmima. Neformalno rečeno, algoritam mora biti takav da ga *mora biti u stanju izvesti mašina*.

Sada dolazimo do ključnog problema u definiciji algoritma. Šta zapravo znači “niz koraka koji se može izvesti potpuno mehanički”? Rasprave o ovome su beskrajne i još uvijek nije nađena potpuno prihvatljiva definicija. Zbog toga su formirani razni apstraktni modeli tzv. **univerzalnih računskih mašina**. Ovi modeli su uvedeni još prije nego što su se pojavile stvarne računске mašine (računari). Tako je **A. Turing** uveo pojam **Turingove mašine**, **E. L. Post** uveo je **Postovu mašinu**, dok je **J. Lambek** uveo **beskonačni abakus**. Sve su ovo primjeri mašina koje su sposobne da obavljaju sve radnje bez primjene ikakve intuicije ili inteligencije, a koje *izgleda* da su dovoljno moćne da obave sve postupke koje je čovjek u stanju da obavi čisto mehanički, dakle bez pozivanja na inteligenciju, inspiraciju ili intuiciju. Pored toga, cilj je da ove mašine budu što je god moguće jednostavnije (bez ikakvih nepotrebnih komplikacija), sa ciljem da matematski aparat koji ih opisuje bude što je god moguće prostiji, a da ipak budu dovoljno moćne da mogu simulirati sve poznate aspekte ljudskog izračunavanja koje se ne zasniva na inteligenciji, inspiraciji ili intuiciji. O ovim mašinama ćemo detaljnije govoriti nešto kasnije. Na ovom mjestu ćemo samo u najgrubljim crtama opisati *Turingovu mašinu*, s obzirom da nam je rudimentalno poznavanje njenih svojstava neophodno za izlaganja koja neposredno slijede, dok ćemo se na njen detaljniji opis vratiti uskoro.

Turingova mašina se može opisati kao čisto apstraktni matematički model, ili kao konkretni model mašine koji se zaista može fizički realizirati. Radi boljeg razumijevanja, prvo ćemo razmotriti ovaj drugi model, prema kojem je Turingova mašina sačinjena iz magnetne trake neograničene dužine, na kojoj mogu biti upisani simboli nekog *unaprijed definiranog alfabeta sa konačno mnogo simbola* (recimo binarnih cifara 0 i 1, cifara dekadnog brojnog sistema ili slova abecede) kao i *glave koja može pročitati simbol na traci* i eventualno ga *zamijeniti drugim simbolom iz istog alfabeta*. Glava se može *pomijerati nalijevo ili nadesno*, za poziciju koju zauzima jedan simbol alfabeta (ponekad se u modelu Turingove mašine umjesto pokretne glave razmatra pokretna traka, no to je suštinski nebitno). Dalje, uzima se da se mašina može nalaziti u *konačno mnogo različitih stanja* od kojih zavisi akcija mašine nakon što se na određenom mjestu pročita određeni simbol. Također, mašina može *prelaziti iz jednog u drugo stanje, ovisno o zatečenom stanju i pročitanim simbolu*. Ponašanje mašine u potpunosti je definirano upotrijebljenim alfabetom, kao i konačnom *tablicom prelaza*, koja definira akcije koje mašina može vršiti ukoliko u nekom od zatečenih stanja pročita određeni simbol (ove akcije nazivamo **instrukcijama** Turingove mašine). Moguće akcije su zamjena simbola drugim simbolom, ili pomjeranje glave ulijevo odnosno udesno. Svaka akcija praćena je prelaskom mašine u novo stanje, koje može biti i identično već zatečenom stanju. U grubim crtama, to je cjelokupan opis Turingove mašine. Cijela ideja Turingove mašine zapravo je zasnovana na oponašanju šta čovjek radi kada izvodi računanja sa papirom i olovkom. Sve se zapravo svodi na pisanje nekih simbola i dopisivanje novih ili brisanje postojećih ovisno od toga šta se već nalazi na papiru, kao i od izvjesnih *stanja* koja opisuju do koje etape smo stigli u procesu računanja.

Primijetimo da se opisani koncept Turingove mašine zaista može realizirati kao konkretan fizički ostvarljiv uređaj, ako zanemarimo nerealističan zahtjev za trakom neograničene dužine, koji se može realizirati samo aproksimativno. U suštini, ograničenje na konačnu dužinu trake samo će ograničiti veličinu problema koji se može rješavati pomoću mašine, koje se za svaki konkretan problem može prevazići uzimanjem duže trake (tj. povećavanjem “količine radne memorije”). Bez obzira na činjenicu da se na konceptu Turingove mašine može zasnovati fizički ostvarljiva “računska mašina”, ni jedan stvarni računar nije zasnovan na tom principu, s obzirom da se Turingova mašina nije pokazala kao “praktično” sredstvo računanja, sa aspekta efikasnosti rješavanja problema.

Razmatrane su brojne modifikacije Turingove mašine koje na prvi pogled djeluju mnogo moćnije od prvobitne varijante, recimo modifikacije koje koriste više glava i više traka, zatim modifikacije u kojima se glava može pomijerati u jednom koraku za više mjesta, a ne samo za jedno mjesto u jednom koraku, itd. Međutim, pokazalo se da je svaka predložena modifikacija *logički ekvivalentna* sa izvornim modelom Turingove mašine, odnosno da izvorno zamišljena Turingova mašina može efektivno *oponašati (simulirati)* svaku svoju predloženu modifikaciju. Jedine razlike mogu biti u *efikasnosti*, odnosno neka mašina koja je “moćnija” od izvorne Turingove mašine može riješiti isti problem mnogo efikasnije (brže), odnosno u manjem broju koraka. Također, bez obzira na činjenicu da razni drugi predloženi modeli univerzalnih računskih mašina poput Postove mašine i beskonačnog abakusa nemaju gotovo nikakvih sličnosti sa Turingovom mašinom, pokazano je da je *ma koja od njih u stanju da oponaša rad ma koje druge*. Zapravo, do današnjeg dana nije poznat *niti jedan model univerzalne računске mašine (uključujući i računare koji se danas proizvode kao stvarni računari)* čiji se računski aspekti ne bi mogli oponašati na Turingovoj mašini,

ili nekoj njoj ekvivalentnoj mašini (poput Postove mašine, ili beskonačnog abakusa). To znači da nije poznat niti jedan problem koji bi se mogao riješiti uz pomoć ma kojeg od predloženih modela računskih mašina koji se ne bi mogao riješiti uz pomoć Turingove mašine. Inače, hipoteza koja tvrdi da se rad *svake izvodljive računske mašine može simulirati na Turingovoj mašini*, poznat je kao **Turingova teza**. Ipak, obično se danas kao model računske mašine za potrebe teoretskih razmatranja uzima upravo Turingova mašina, s obzirom da je od svih modela ona najviše proučavani model, odnosno najviše poznatih teoretskih rezultata izvedeno je upravo na modelu Turingove mašine.

S obzirom da se Turingova mašina može definirati čisto formalno kao odgovarajuća algebarska struktura (o čemu ćemo kasnije nešto više reći), bez ikakvog ulaženja u njenu fizičku strukturu, ona može poslužiti za *formalnu definiciju algoritma*, koja je posve matematički precizna, bez ikakvih nejasnoća i nepreciznosti. Naime, algoritam možemo definirati kao *konačan niz instrukcija Turingove mašine koji nakon konačno mnogo primijenjenih instrukcija na osnovu ulaznih podataka pohranjenih na traci koji predstavljaju opis problema, zapisuje na traku rješenje problema ili odgovor da rješenja nema*. S obzirom na mogućnost međusobnog oponašanja Turingove mašine i drugih predloženih modela univerzalnih računskih mašina, slične definicije mogu se zasnovati i na drugim modelima računskih mašina, recimo Postovoj mašini, ili beskonačnom abakusu. Ipak, definicija zasnovana na Turingovoj mašini najčešće se koristi.

Tokom razvoja teorije izračunljivosti, javili su se i brojni pokušaji da se pojam algoritma formalizira bez pozivanja na neki model univerzalne računske mašine. Naime, mnogi su smatrali da su koncepti univerzalnih računskih mašina *isuviše ograničeni*, i da bi se čisto matematičkim jezikom možda mogli formalizirati neki postupci koji su *intuitivno izvodljivi čisto mehanički*, a koje predloženi modeli računskih mašina možda nisu u stanju izvesti. Tako je **S. Kleene** razvio model **rekurzivnih funkcija** kao jedan od prvih modela koji je zamišljen kao formalizacija pojma algoritma. **E. L. Post** je razvio **Postove sisteme zamjene** (engl. *rewriting systems*), **A. Markov** takozvane **normalne algoritme**, dok je **A. Church** razvio **λ -račun**, vjerovatno najapstraktniji od svih pokušaja formalizacije pojma algoritma (na sve ove pomenute modele ukratko ćemo se osvrnuti nešto kasnije). Mada svi ovi modeli nisu ni nalik jedan drugom, niti su nalik nekom od modela univerzalnih računskih mašina, svi oni su se pokazali logički ekvivalentnim u smislu da se bilo koji od ovih modela može mehanički izvesti na bilo kojem od predloženih modela univerzalnih računskih modela, kao i da se svaki predloženi model univerzalne računske mašine može formalno prevesti u ma koji od predloženih "nemašinskih" modela. Dakle, imamo iznenađujuću činjenicu da se *ogroman broj drastično različitih formalnih definicija algoritma na kraju pokazao logički posve ekvivalentnim*.

Izložene činjenice bile su povod za pretpostavku, poznatu pod nazivom **Church-Turingova teza**, koja tvrdi da je *ma kakav postupak koji je izvodljiv čisto mehanički u ma kakvom intuitivnom smislu, izvodljiv i na Turingovoj mašini* (odnosno, pomoću bilo kojeg od njoj ekvivalentnih modela). Ovo je zapravo *pojačana verzija Turingove teze*. Ukoliko prihvatimo ovu tezu, one probleme za koje ni teoretski ne postoji algoritam za njihovo rješavanje (ukoliko pod algoritmom smatramo ono što je izvodljivo na Turingovoj mašini), ne može riješiti ni čovjek. Do sada nema nikakvih konkretnih evidencija (osim onih čisto psihološke prirode) koje bi osporile ovu tezu, odnosno sve do sada prikupljene činjenice idu joj u prilog. Donekle nezgodna posljedica ove teze je što, ukoliko je ona tačna, iz nje slijedi da čovjek *razmišlja čisto algoritamski*. Doduše, moguće je da algoritmi koje koristi čovjek mogu biti enormno složeni, tako da se ne mogu efektivno iskazati.

Provedena razmatranja vode nas do interesantnog filozofskog pitanja *da li je moguće mehanizirati postupke ljudskog mozga*. Jasno je da je, ukoliko vrijedi Church-Turingova teza, odgovor pozitivan. Zapravo, sumnja u pozitivan odgovor na ovo pitanje ujedno je i *jedini razlog zbog kojeg neki sumnjaju u ovu tezu* (s obzirom da nikakva druga evidencija koja joj ne bi išla u prilog nije poznata). Generalno, po ovom pitanju javile su se dvije struje. Takozvana struja **jake vještačke inteligencije** (engl. *strong artificial intelligence*) tvrdi da mehanizirani algoritam koji odgovara postupcima ljudskog mozga *postoji*, mada je *slabo vjerovatno da će se on ikada moći sintetizirati*, zbog svoje *enormne složenosti*. Ovo mišljenje je danas dominantno. Suprotna struja, poznata kao struja **slabe vještačke inteligencije** (engl. *weak artificial intelligence*) zastupa tvrđenje da je moguće algoritamski oponašati samo *izvjesne aspekte ljudskog mozga*, ali da ljudsko mišljenje u cjelini *nije moguće algoritmizirati*, jer se nalazi *izvan svih do sada poznatih modela*. S obzirom da se svi dosada poznati (i primijenjeni) zakoni fizike mogu algoritmizirati, ovakvo tvrđenje povlači da mozak u

svom radu *izlazi izvan okvira poznatih zakona fizike*. Izraziti pobornik ove struje je matematičar i fizičar **R. Penrose**, koji u svojoj knjizi *Emperor's New Mind* sugerira da se rad mozga u izvjesnom dijelu oslanja na (još neotkrivene) zakone kvantne teorije gravitacije, odnosno da mozak neophodne informacije za nealgoritamsko procesiranje crpi iz interakcija sa kvantnim gravitacionim poljem (tačnije hipotetičkim česticama nazvanim **gravitoni**). Ova teorija je veoma špekulativna, mada ne i posve neosnovana. Kasnije se u pokušaju da pronađe biološke dokaze za svoje špekulacije, Penrose povezao sa anesteziologom **S. Hameroffom**. Oni su iznijeli špekulaciju da se te interakcije dešavaju u dijelu moždanih ćelija koje se nazivaju **mikrotubule**, odnosno da su one "prijemnici" dodatnih informacija koje se crpe iz kvantnog gravitacionog polja. Njihova zajednička teorija poznata je pod nazivom **orkestrirana objektivna redukcija** (skraćeno **Orch-OR**). U posljednje vrijeme, njih dvojica su sa svojim špekulacijama otišli toliko daleko, da tvrde da su dokazali čak i postojanje **duše**, pri čemu su dušu povezali sa (neuništivim) informacijama o kvantnim stanjima pohranjenim unutar mikrotubula. Koliko god njihove teorije djelovale interesantno, činjenica je da većina fizičara njihove teorije (pogotovo ove posljednje) svrstava u polje čistih "lupetanja".

Egzistencija algoritamski nerješivih problema

Nakon što smo precizirali pojam algoritma, prirodno je postaviti pitanje da li za svaki problem postoji algoritam koji vodi ka njegovom rješenju (odnosno odgovoru da rješenje ne postoji). Na žalost, *odgovor je određen*. Zbog toga ćemo reći da je problem **algoritamski rješiv** ako je rješiv prema bilo kojem od pomenutih modela algoritama (recimo, na Turingovoj mašini). Konkretnije, reći ćemo da je problem algoritamski rješiv ukoliko postoji *skup instrukcija* za Turingovu mašinu takav da Turingova mašina koja radi prema tom skupu instrukcija uz ulazne podatke zapisane na traci nakon konačno mnogo primijenjenih instrukcija prestaje sa radom, pri čemu rješenje problema (ili informacija o nepostojanju rješenja) ostaje zapisano na traci. U suprotnom, kažemo da je problem **algoritamski nerješiv**. Bitno je naglasiti da algoritamska nerješivost znači da je *dokazano* da algoritamsko rješenje ne postoji, a ne da ono prosto nije poznato, ili da nije do sada nađeno.

Činjenica da postoje algoritamski nerješivi problemi na prvi pogled je neobična. Međutim, lako ćemo se uvjeriti da takvi problemi *moraju postojati*. Pojam algoritamske rješivosti tijesno je povezan sa pojmom **izračunljivosti**, odnosno pojmom **izračunljivih funkcija**. Radi jednostavnosti, ograničićemo se samo na funkcije sa domenom i kodomenom u skupu $\mathbb{N}$, mada se uvedeni pojmovi mogu generalizirati i na opsežnije domene. Za funkciju $f: \mathbb{N} \rightarrow \mathbb{N}$ kažemo da je **izračunljiva** (engl. *computable*) ukoliko *postoji algoritam za njeno računanje*, odnosno algoritam koji za ma kakvu ulaznu vrijednost n računa vrijednost $f(n)$. Ukoliko takav algoritam ne postoji, za funkciju kažemo da je **neizračunljiva** (engl. *noncomputable*).

Postojanje neizračunljivih funkcija direktno slijedi iz činjenice da je skup svih algoritama *prebrojiv* (neovisno od toga za kakvu se definiciju algoritma odlučimo), dok je skup svih funkcija sa $\mathbb{N}$ u $\mathbb{N}$ *neprebrojiv* (ovo na neki način znači da ima neuporedivo više funkcija sa $\mathbb{N}$ u $\mathbb{N}$ nego što ima algoritama). Izložićemo prvo neformalni verbalni dokaz tvrdnje da je skup svih algoritama prebrojiv (ovaj dokaz se, po potrebi, može lako formalizirati). Naime, ukoliko je algoritam konačan skup koraka, on se sigurno može opisati kao *konačan niz rečenica (instrukcija)* koje se zapisuju znakovima iz nekog *konačnog skupa znakova (alfabeta)*. Dakle, svi algoritmi su opisivi kao *stringovi konačne dužine nad konačnim alfabetom*, a skupovi svih takvih stringova su prebrojivi. Zaista, ukoliko alfabet ima N različitih znakova, svaki string nad tim alfabetom možemo posmatrati kao prirodan broj zapisan u bazi N , pri čemu različitim stringovima odgovaraju različiti prirodni brojevi. Slijedi da svakom algoritmu možemo jednoznačno pridružiti neki prirodan broj, odnosno postoji jednoznačno preslikavanje iz skupa algoritama u skup $\mathbb{N}$, iz čega slijedi prebrojivost skupa svih algoritama. S druge strane, neprebrojivost skupa svih funkcija sa $\mathbb{N}$ u $\mathbb{N}$ lako se pokazuje *Cantorovim dijagonalnim postupkom*. Zaista, kada bi skup svih funkcija sa $\mathbb{N}$ u $\mathbb{N}$ bio prebrojiv, tada bi bilo moguće sve funkcije sa $\mathbb{N}$ u $\mathbb{N}$ poredati u niz $f_1, f_2, f_3, \dots$ takav da se u njemu nalaze *sve funkcije* sa $\mathbb{N}$ u $\mathbb{N}$. Posmatrajmo sada funkciju $g: \mathbb{N} \rightarrow \mathbb{N}$ takvu da je $g(n) \neq f_n(n)$ za svako $n \in \mathbb{N}$. Recimo, možemo uzeti da je $g(n) = f_n(n) + 1$. Takva funkcija g očigledno ne može pripadati nizu $f_1, f_2, f_3, \dots$ jer se od svake funkcije u nizu razlikuje bar za jednu vrijednost argumenta (recimo, od funkcije f_2 se sigurno razlikuje za $n = 2$), što je u suprotnosti sa pretpostavkom da niz $f_1, f_2, f_3, \dots$ sadrži sve funkcije sa $\mathbb{N}$ u $\mathbb{N}$. Iz ove kontradikcije slijedi da skup svih funkcija sa $\mathbb{N}$ u $\mathbb{N}$ *nije prebrojiv*.

Iz izloženog slijedi da je skup svih funkcija sa $\mathbb{N}$ u $\mathbb{N}$ "obimniji" od skupa svih algoritama, tako da je jasno da moraju postojati funkcije sa $\mathbb{N}$ u $\mathbb{N}$ takve da *ne postoji algoritam za njihovo računanje!* Doduše, s obzirom na očiglednu *nekonstruktivnu prirodu* izloženog dokaza, dugo se mislilo da su neizračunljive funkcije *čisto matematička egzotika*, odnosno da nije moguće stvarno konstruisati neku *konkretnu* funkciju sa $\mathbb{N}$ u $\mathbb{N}$ koja bi bila neizračunljiva. Međutim, to se nažalost pokazalo kao *netačno*. Danas postoji dosta značajnih konkretnih primjera precizno definiranih ali neizračunljivih funkcija sa $\mathbb{N}$ u $\mathbb{N}$, i to takvih da bi bilo jako korisno kada bi one bile izračunljive. Sa nekim od poznatih primjera neizračunljivih funkcija susrešćemo se nešto kasnije.

Kao što je već rečeno, postoje brojni problemi za koje je dokazano da su algoritamski nerješivi. Jedan od prvih problema čija je nerješivost dokazana, tačnije *drugi po redu* (mada ne i prvi, kako se ponegdje pogrešno tvrdi) je takozvani **problem zaustavljanja**, poznat i pod imenom **Halting problem**. Ovaj problem se sastoji u ispitivanju *da li neki zadani program (napisan u nekom programskom jeziku) zaustavlja rad nakon konačnog vremena*, odnosno da li zadani program *terminira ili ne*. Općenitije, treba utvrditi *da li neki proizvoljno zadani niz koraka terminira ili ne* (tj. da li ispunjava jedan od temeljnih uvjeta da bi bio algoritam). Već smo vidjeli da odgovor na to pitanje u općem slučaju nije nimalo jednostavan (npr. za ranije spominjani *Collatzov proces* odgovor do danas nije poznat). Dakle, problem zaustavljanja sastoji se u tome da se pronađe algoritam (tj. efektivni mehanički izvodljiv postupak) koji će utvrditi da li neki zadani niz koraka koji se može izvesti čisto mehaničkim putem terminira ili ne, tj. da li predstavlja algoritam ili ne (i koji bi, recimo, mogao utvrditi da li Collatzov proces terminira za sve ulazne podatke ili ne). Uskoro ćemo pokazati da takav algoritam naprosto *ne postoji*, odnosno da problem zaustavljanja *nije algoritamski rješiv*.

Razmotrimo jedan praktičan značaj Halting problema. Primjetimo da se svaka teorema aritmetike (pa i mnogih drugih oblasti matematike) može napisati u predikatskoj formi kao $(\forall x \in \mathbb{N}^n) P(x)$ odnosno $(\exists x \in \mathbb{N}^n) P(x)$, gdje je x neka uređena n -torka brojeva, a $P(x)$ logički izraz (predikat), koji kao promjenljive sadrži elemente n -torke x , a pored toga može sadržavati i konstante, aritmetičke operatore (npr. "+"), kao i relacije (npr. "<") i logičke (npr. "&") operatore. Naprimjer, čuvena *velika Fermatova teorema* može se lako prikazati u obliku $(\forall (x_1, x_2, x_3, x_4) \in \mathbb{N}^4) (x_4 > 2 \Rightarrow x_1^{x_4} + x_2^{x_4} \neq x_3^{x_4})$. Odavde slijedi da bi rješivost Halting problema omogućila *automatsko dokazivanje teorema aritmetike*. Na primjer, za jednodimenzionalni slučaj, odnosno slučaj kada se n -torka x sastoji samo od jedne promjenljive, bilo bi dovoljno ispitati da li sljedeći program terminira ili ne (korištena je sintaksa jezika C odnosno C++, što nimalo ne umanjuje općenitost razmatranja):

```
int x = 1;  
while (P(x)) x++;
```

Lako se vidi da ukoliko ovaj program ne terminira, slijedi da je teorema $(\forall x \in \mathbb{N}) P(x)$ tačna, dok u suprotnom nije. Ovaj primjer nije teško generalizirati za višedimenzionalne slučajeve, kao i za slučajeve u kojima se umjesto kvantifikatora "∀" javlja kvantifikator "∃", s obzirom da je iskaz $(\exists x \in \mathbb{N}^n) P(x)$ svakako ekvivalentan iskazu $\neg(\forall x \in \mathbb{N}^n) \neg P(x)$. Činjenica da promjenljive tipa "int" imaju ograničen opseg tako da ne pokrivaju čitav skup prirodnih brojeva također ništa ne umanjuje općenitost razmatranja, jer uvijek možemo zamisliti da je program napisan u nekom programskom jeziku koji koristi sintaksu jezika C ili C++, ali u kojem *opseg cjelobrojnih promjenljivih nije ograničen*. Recimo, u jeziku C++ bismo umjesto tipa "int" mogli koristiti neki korisnički definirani tip podataka (klasu), koji podržava rad sa brojevima proizvoljne veličine, i za koji je definiran operator "++" i svi operatori koji se javljaju u predikatu P.

Iz izloženog slijedi da bi rješivost Halting problema omogućila jednostavno automatsko dokazivanje ili opovrgavanje *svih* teorema aritmetike (kao i mnogih drugih oblasti matematike). Nažalost, Halting problem je nerješiv, a to se jednostavno pokazuje varijacijom čuvenog *paradoksa brijača*, koji smo već spominjali (i koji zapravo predstavlja šaljivo iskazanu varijantu *Russellovog paradoksa* iz teorije skupova). Prvo ćemo, radi lakšeg praćenja, koristiti sintaksu jezika C++, a nakon toga ćemo dati općenitiji dokaz ne oslanjajući se ni na jedan konkretan programski jezik, ili još općenitije, ne oslanjajući se ni na kakav konkretan model pomoću kojeg je algoritam iskazan. Pretpostavimo da imamo funkciju pod nazivom "zaustavlja_se" koja ispituje da li se neki program pohranjen u datoteci čije se ime zadaje kao argument

zaustavlja u konačnom vremenu, i vraća logičku vrijednost "tačno" ili "netačno" (odnosno "true" ili "false") kao rezultat. Dakle, kostur funkcije "zaustavlja_se" mogao bi izgledati ovako:

```
bool zaustavlja_se(string ime){  
    ...  
}
```

Neka, s druge strane, imamo program koji koristi funkciju "zaustavlja_se", i koji izgleda ovako:

```
int main() {  
    if(zaustavlja_se("abc.cpp")) while(true);  
    return 0;  
}
```

Jasno je da ovaj program terminira ako i samo ako program snimljen pod imenom "abc.cpp" ne terminira. Paradoks se javlja ukoliko upravo ovaj program snimimo pod imenom "abc.cpp" i pokrenemo ga, jer ćemo doći do zaključka da on terminira ako i samo ako on ne terminira, što je očigledna protivrječnost. Ovo nas dovodi do jedinog mogućeg zaključka da funkcija poput funkcije "zaustavlja_se" *ne može postojati*.

Izloženi dokaz lako možemo uopćiti tako da se oslobodimo zavisnosti od sintakse jezika C++. Pretpostavimo da postoji algoritam (nazovimo ga H) koji je u stanju da odredi da li neki mehanički izvodljivi niz koraka (predstavljen na neki način, recimo kao niz simbola nad nekim alfabetom) terminira ili ne kada mu se kao ulaz ponudi neki ulazni podatak. Formirajmo sada niz koraka T koji se također izvodi nad nekim nizom koraka S na sljedeći način. Niz koraka T prvo poziva algoritam H sa ciljem da utvrdi da li niz koraka S terminira kada mu se kao ulaz ponudi sam niz koraka S. Ukoliko je odgovor potvrđan, niz koraka T upada u beskonačnu petlju, inače se završava. Do paradoksa sada dolazi ukoliko primijenimo niz koraka T nad samim sobom (niz koraka T primijenjen nad samim sobom terminira ako i samo ako ne terminira). Iz ovog paradoksa slijedi da algoritam H ne može postojati.

Već smo rekli da se algoritamska nerješivost Halting problema negativno odražava na mogućnost automatskog (algoritamski izvodljivog) dokazivanja teorema aritmetike. Doduše, samo po sebi, algoritamska nerješivost Halting problema još uvijek ne znači da problem automatskog dokazivanja teorema aritmetike nije možda algoritamski rješiv na neki drugi način koji se ne poziva na Halting problem. Međutim, nade u takvu mogućnost su se definitivno raspršile kada je pokazano da se *sama formulacija Halting problema može napisati u vidu teoreme aritmetike* (za tu svrhu se koristi *formalna definicija Turingove mašine*, ili *Kleeneov model rekurzivnih funkcija*). Stoga, kada bi sve teoreme aritmetike bile dokazive algoritamskim putem, bio bi rješiv i Halting problem, a znamo da on to nije. Slijedi da i problem automatskog dokazivanja teorema aritmetike također nije algoritamski rješiv.

Nerješivost Halting problema još uvijek ne znači da nije moguće sastaviti algoritam koji za *neke nizove koraka* daje odgovor da li terminiraju ili ne. Zapravo, moguće je sastaviti algoritam koji za *sve nizove koraka koji terminiraju sigurno daje odgovor da oni terminiraju*, ali tako nešto nije moguće za *sve nizove koraka koji ne terminiraju* (mada za neke jeste). Pretpostavimo da imamo neki takav *polovičan* algoritam koji radi za neke slučajeve, a za neke ne radi (recimo, ponekad daje odgovor da *ne može utvrditi* da li zadani niz koraka terminira ili ne). Interesantno je da je uvijek moguće njegovom analizom konstruisati niz koraka koji *sigurno ne terminira*, a za koji predloženi algoritam *ne može dati odgovor* (ova konstrukcija se čak može izvesti mehanički, tj. algoritamski). Pokažimo kako se to može izvesti. Neka je H neki algoritam koji *djelimično rješava* Halting problem, tj. daje potvrđan odgovor kad god neki zadani mehanički postupak terminira kada mu se kao ulaz ponudi neki ulazni podatak, dok u suprotnom ponekad daje negativan odgovor, a ponekad daje odgovor da *ne zna* da li postupak terminira ili ne. Formirajmo sada algoritamski proces (niz koraka) T koji se izvodi nad nekim drugim nizom koraka S na sljedeći način. Niz koraka T prvo poziva algoritam H sa ciljem da utvrdi da li niz koraka S terminira kada mu se kao ulaz ponudi sam niz koraka S. Ukoliko H ponudi potvrđan ili neodređen odgovor, niz koraka T upada u beskonačnu petlju, inače se završava. Ispitajmo sada šta se dešava ukoliko se niz koraka T pozove nad *samim sobom*, odnosno da li se on završava ili ne. Pretpostavimo da se on završava. On prvo poziva algoritam H sa ciljem da utvrdi da li T terminira kada mu se kao ulaz ponudi sam niz koraka T.

Pošto smo pretpostavili da H sigurno daje potvrđan odgovor kad god niz koraka koji testira zaista terminira, H će tada dati potvrđan odgovor. Međutim, iz opisa niza koraka T , slijedi da T tada upada u beskonačnu petlju, odnosno ne terminira. Dakle, pretpostavka da T terminira kada mu se ponudi sam niz koraka T kao ulaz dovela nas je do zaključka da T ne terminira pod istim pretpostavkama. Kako je ovo očita kontradikcija, slijedi da T *ne terminira* kada mu se ponudi sam niz koraka T kao ulaz. Međutim, algoritam H ovo *nije u stanju da utvrdi*. Zaista, pretpostavimo da je H u stanju da utvrdi da niz koraka T ne terminira kada mu se ponudi on sam kao ulaz. U tom slučaju, kada pozovemo niz koraka T nad samim sobom, prvo bi se pozvao algoritam H da utvrdi da li T terminira kada mu se kao ulaz ponudi on sam. Međutim, ukoliko bi H dao negativan odgovor, prema opisu niza koraka T , slijedi da tada T završava sa radom, odnosno on terminira, što je ponovo kontradikcija. Dakle, pretpostavka da je H u stanju utvrditi da niz koraka T ne terminira kada mu se ponudi on sam kao ulaz dovodi do kontradikcije, iz čega jasno slijedi da H nije u stanju utvrditi tu činjenicu. Dakle, kakav god bio algoritam H , konstruisali smo niz koraka za koji *znamo da se ne zaustavlja*, ali koji je takav da *algoritam H to ne može detektirati*.

Jasno je da je nakon konstrukcije niza koraka sa opisanim svojstvima moguće proširiti algoritam tako da daje određen odgovor i za tako konstruisani niz koraka. Međutim, za tako prošireni algoritam ponovo je moguće istim postupkom konstruisati niz koraka koji ne terminira, a za koji tako prošireni algoritam ne može dati odgovor. Ovaj proces može se nastaviti unedogled. Slijedi da je moguće konstruisati čitav beskonačan niz sve boljih (i sve složenijih i složenijih) algoritama, koji su primjenljivi na sve više i više slučajeva, ali da nikada ne možemo postići da algoritam bude primjenljiv na *sve slučajeve*. Pokazuje se da ovo ne vrijedi samo za Halting problem, nego i za većinu drugih problema koji su algoritamski nerješivi. Tako, recimo, postoje algoritmi koji mogu automatski dokazivati (ili opovrgavati) *mnoge* teoreme aritmetike, *ali ne sve*. Štaviše, za svaki takav algoritam je moguće (i to čisto algoritamskim putem) konstruisati (tačnu) teoremu aritmetike koju taj algoritam nije u stanju dokazati.

Bitno je istaći da se u definiciji algoritamske rješivosti nekog problema (odnosno izračunljivosti neke funkcije) zahtijeva samo da *postoji algoritam* koji rješava taj problem (odnosno računa tu funkciju) čak i onda kada mi *ne znamo koji je to algoritam* (a možda čak i nikada nećemo znati). Na primjer, može se desiti da imamo nekoliko algoritama za koje znamo da *jedan od njih sigurno rješava neki problem*, ali *ne znamo tačno koji*. Takvi problemi su, prema definiciji, algoritamski rješivi, bez obzira što mi *ne znamo* algoritam za njihovo rješavanje. Navedimo dva karakteristična primjera za ovo. Neka je data funkcija

$$f(n) = \begin{cases} 1, & \text{ukoliko u decimalnom zapisu broja } \pi \text{ postoji } n \text{ uzastopnih nula} \\ 0, & \text{u suprotnom} \end{cases}$$

Kako trenutno stvari stoje, mi nemamo dovoljno znanja o decimalnom zapisu broja π da bismo mogli izračunati vrijednost ove funkcije za proizvoljan argument n . Recimo, zna se da je $f(3) = 1$ jer u decimalnom zapisu broja π zaista postoje mjesta gdje imamo 3 uzastopne nule, ali ne zna se koliko je $f(1000)$, jer niko ne zna da li u decimalnom zapisu broja π igdje ima 1000 uzastopnih nula. Bez obzira na to, ova funkcija je ipak *izračunljiva*. Zaista, postoje samo dvije mogućnosti. Jedna je da u decimalnom zapisu broja π postoje proizvoljno duge sekvence uzastopnih nula. U tom slučaju, vrijednost funkcije f jednaka je jedinici za *svaku vrijednost argumenta n* , tako da u tom slučaju trivijalni algoritam koji stalno vraća jedinicu kao rezultat računa tu funkciju. Druga mogućnost je da u decimalnom zapisu broja π postoji najviše k uzastopnih nula, gdje je k neki broj. U tom slučaju je $f(n) = 1$ za $n \leq k$, a $f(n) = 0$ za $n > k$, tako da algoritam koji vraća jedinicu za $n \leq k$ a nulu za $n > k$ računa ovu funkciju. U svakom slučaju, algoritam za računanje ove funkcije *postoji*, samo što mi *ne znamo koji je to tačno algoritam*.

Sljedeći primjer je još interesantniji, jer dok za prethodnu funkciju možda nekada i saznamo (ukoliko saznamo više informacija o broju π) koji je tačno algoritam računa, za sljedeću funkciju teško da ćemo ikada pouzdano saznati koji je algoritam računa (barem ne dok smo živi). Neka je data funkcija

$$f(n) = \begin{cases} 1, & \text{ukoliko Bog postoji} \\ 0, & \text{ukoliko Bog ne postoji} \end{cases}$$

Djeluje šokantno, ali ova funkcija je *također izračunljiva*! Zaista, primijetimo da ova funkcija zapravo i ne ovisi od svog argumenta n (odnosno, ona je konstantna funkcija). Ukoliko Bog postoji, ova funkcija stalno

daje vrijednost 1, te algoritam koji vraća jedinicu kao rezultat neovisno od n računa ovu funkciju. S druge strane, ukoliko Bog ne postoji, ova funkcija stalno daje vrijednost 0, te algoritam koji vraća nulu kao rezultat neovisno od n računa ovu funkciju. Dakle, jedan od pomenuta dva algoritma sigurno računa ovu funkciju, bez obzira *što ne znamo koji*. Činjenica da će vjernici ubijedeno tvrditi da je to prvi algoritam, a ateisti da je to drugi algoritam (dok će agnostici i skeptici ipak tvrditi da ne znaju) ništa ne mijenja situaciju, odnosno činjenicu da algoritam koji računa ovu funkciju *postoji*, te je ona *izračunljiva*.

Za dokazivanje da je neki problem algoritamski nerješiv (odnosno da je neka funkcija neizračunljiva) uglavnom se koriste dva pristupa. Prvi pristup je da se pretpostavi da postoji algoritam za rješavanje tog problema (odnosno računanje te funkcije), te se pokaže da iz te pretpostavke slijedi neka kontradikcija, što zahtijeva da odbacimo pretpostavku. Taj smo pristup koristili recimo pri dokazivanju algoritamske nerješivosti Halting problema. Drugi pristup je da se pokaže da bi iz algoritamske rješivosti nekog problema slijedila i algoritamska rješivost nekog drugog problema, za koji smo ranije pokazali da je algoritamski nerješiv. To onda povlači da ni taj problem ne može biti algoritamski rješiv. Taj pristup je recimo iskorišten da se pokaže da je problem dokazivanja teorema aritmetike algoritamski nerješiv, jer je pokazano da bi njegova rješivost povlačila i rješivost Halting problema, za koji je ranije utvrđeno da je algoritamski nerješiv. Kasnije ćemo se upoznati sa još nekim značajnim primjerima algoritamski nerješivih problema i neizračunljivih funkcija, kao i sa osnovnim idejama kako je pokazana njihova algoritamska nerješivost odnosno neizračunljivost.

Turingova mašina kao model univerzalne računske mašine

Sada ćemo malo detaljnije razmotriti Turingovu mašinu kao model univerzalne računske mašine. Postoji mnogo različitih načina kako se u literaturi *formalno* definira Turingova mašina (pri čemu se pod formalnom definicijom smatra čisto matematička definicija koja ni na kakav način ne implicira fizičku strukturu mašine). Bez obzira na pojedine razlike, sve formalne definicije načelno se slažu u tome da se Turingova mašina sastoji od **alfabeta** Γ , koji predstavlja *konačan skup simbola*, i koji obavezno sadrži jedan specijalan simbol nazvan **praznina** (engl. *blank*) koji ćemo obilježavati sa " $\square$ ", zatim konačnog **skupa stanja** Q , **početnog stanja** $q_0 \in Q$ i **tablice instrukcija** (odnosno **tablice prelaza**) Δ koja se sastoji od uređenih četvorki oblika $(q', \gamma', q'', \gamma'')$, pri čemu je $q' \in Q$, $q'' \in Q$, $\gamma' \in \Gamma$ i $\gamma'' \in \Gamma \cup \{\leftarrow, \rightarrow\}$, gdje su " $\leftarrow$ " i " $\rightarrow$ " dva specijalna simbola koji se *ne nalaze* unutar alfabeta Γ (intuitivno, ovim simbolima odgovaraju operacije pomjeranja glave ulijevo odnosno udesno). Pored toga, na četvorke $(q', \gamma', q'', \gamma'')$ se postavlja dodatni uvjet da nikoje dvije četvorke iz Δ nemaju međusobno jednake prve dvije koordinate. Alternativno, Δ se nekada definira i kao *funkcija* $\Delta: Q \times \Gamma \rightarrow Q \times \Gamma \cup \{\leftarrow, \rightarrow\}$, što obezbjeđuje da je ovaj posljednji uvjet automatski ispunjen. Također se ponekad razlikuje **radni alfabet** Γ i **ulazni alfabet** Σ , koji predstavlja neki podskup od $\Gamma \setminus \{\square\}$. Intuitivno, mašina pri svom radu može koristiti sve simbole iz radnog alfabeta Γ , ali se samo ulazni alfabet može koristiti za zadavanje problema.

Svakoј Turingovoj mašini pridružuje se tzv. **konfiguracija**, koja se sastoji od sekvence t_i , $i \in \mathbb{Z}$ simbola iz Γ , koja se naziva **traka**, pri čemu je samo *konačno mnogo* članova sekvence različito od praznine, zatim **tekućeg stanja** $q \in Q$ i cijelog broja p koji se naziva **pozicija glave**. Dakle, konfiguracija je uređena trojka (t, q, p) . Pod **računanjem** (engl. *computation*) uz pomoć Turingove mašine nazivamo svaki *konačan niz konfiguracija* koji se formira na sljedeći način. U početnoj konfiguraciji imamo $q = q_0$ i $p = 0$, dok sekvenca t_i sadrži *ulazne podatke*, kodirane pomoću simbola iz alfabeta Σ . Podaci su međusobno razdvojeni po jednom prazninom, a njihov zapis počinje počev od indeksa 0 naviše. Svaka sljedeća konfiguracija određuje se na sljedeći način. U tablici instrukcija Δ traži se četvorka $(q', \gamma', q'', \gamma'')$ za koju vrijedi $q' = q$ i $\gamma' = t_p$ (iz uvjeta koji se nameću na Δ slijedi da postoji najviše jedna takva četvorka). Ukoliko takva četvorka ne postoji, tekuća konfiguracija je i posljednja (tj. mašina "završava rad"). U suprotnom, novo tekuće stanje postaje $q = q''$, a dalji tok akcija zavisi od γ'' . Ukoliko je $\gamma'' \in \Gamma$, vrši se zamjena postojećeg simbola t_p sa γ'' , odnosno postaje $t_p = \gamma''$. S druge strane, ukoliko je γ'' jednak simbolu " $\leftarrow$ " odnosno " $\rightarrow$ ", p se smanjuje odnosno povećava za jedinicu (tj. glava se "pomjera" za jedno mjesto "ulijevo" odnosno "udesno"). Proces se dalje nastavlja sve dok ne dođemo do završne konfiguracije. Sekvenca t_i , onakva kakva je u završnoj konfiguraciji, sadrži *rješenje problema*, zapisano pomoću simbola iz alfabeta Γ . Ova izmjena konfiguracija može se precizno opisati pomoću matematičkih formula koje

daju zavisnost naredne konfiguracije od tekuće konfiguracije i elemenata tablice instrukcija Δ . Konkretno, ukoliko tekuću konfiguraciju označimo sa $((t_i)_n, q_n, p_n)$ a sljedeću sa $((t_i)_{n+1}, q_{n+1}, p_{n+1})$, imamo

$$(t_i)_{n+1} = \begin{cases} \gamma & \text{ukoliko je } i = p \text{ i ukoliko u } \Delta \text{ postoji četvorka oblika } (q_n, (t_p)_n, ?, \gamma) \\ (t_i)_n & \text{u suprotnom} \end{cases}$$

$$q_{n+1} = \begin{cases} q & \text{ukoliko u } \Delta \text{ postoji četvorka oblika } (q_n, (t_p)_n, q, ?) \\ q_n & \text{u suprotnom} \end{cases}$$

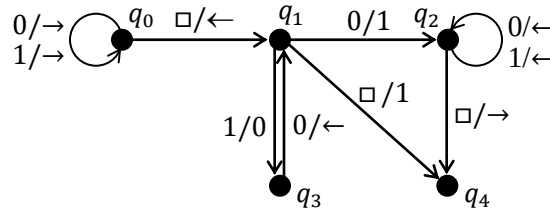
$$p_{n+1} = \begin{cases} p_n + 1 & \text{ukoliko u } \Delta \text{ postoji četvorka oblika } (q_n, (t_p)_n, ?, \rightarrow) \\ p_n - 1 & \text{ukoliko u } \Delta \text{ postoji četvorka oblika } (q_n, (t_p)_n, ?, \leftarrow) \\ p_n & \text{u suprotnom} \end{cases}$$

Upitnik označava da nije bitno šta se nalazi na tom mjestu. Bez obzira na ove formule, prethodno dati verbalni opis je također dovoljno precizan i jasan. Primijetimo da je “redoslijed” navođenja “instrukcija” u tablici instrukcija Δ *posve nebitan*, za razliku od redoslijeda instrukcija u klasičnim programskim jezicima.

Kao primjer, navešćemo konkretnu Turingovu mašinu koja binarni broj upisan na traku povećava za jedinicu. Alfabet ove mašine biće skup $\Gamma = \{0, 1, \square\}$, skup stanja biće $Q = \{q_0, q_1, q_2, q_3, q_4\}$, stanje q_0 će biti početno stanje, dok će tablicu instrukcija Δ sačinjavati sljedeće četvorke:

$(q_0, 0, q_0, \rightarrow)$	$(q_0, 1, q_0, \rightarrow)$	$(q_0, \square, q_1, \leftarrow)$	$(q_1, 0, q_2, 1)$
$(q_1, 1, q_3, 0)$	$(q_1, \square, q_4, 1)$	$(q_2, 0, q_2, \leftarrow)$	$(q_2, 1, q_2, \leftarrow)$
$(q_2, \square, q_4, 1)$	$(q_3, 0, q_1, \leftarrow)$		

Prije nego što razmotrimo kako tačno radi ova Turingova mašina, naglasimo da se tablica instrukcija Turingove mašine može veoma pregledno prikazati pomoću *grafa* u kojem stanjima mašine odgovaraju čvorovi grafa, dok svakoj četvorki iz tablice instrukcija oblika $(q', \gamma', q'', \gamma'')$ odgovara grana koja ide iz čvora pridruženog stanju q' u čvor pridružen stanju q'' označena oznakom γ'/γ'' . Recimo, prikazanoj tablici instrukcija odgovara graf kao na sljedećoj slici:



Mašina započinje rad u stanju q_0 . Pretpostavimo da se na početku na traci nalazi binarni broj koji treba povećati za jedinicu, pri čemu glava pokazuje na prvi znak broja (tj. na najviši bit). Prvo je potrebno da mašina pozicionira glavu na krajnji (najniži) bit broja. Stoga, mašina čita znakove, i sve dok je znak jedinica ili nula, pomjera glavu udesno i ostaje u istom stanju. Kada naiđe na prvi znak koji nije jedinica ili nula, to može biti samo praznina. Mašina pomjera glavu ulijevo i prelazi u stanje q_1 . Povećavanje može da započne. Ukoliko se na mjestu glave nalazi nula, mašina je mijenja sa jedinicom, i prelazi u stanje q_2 . U načelu, povećavanje je tada završeno. Međutim, po konvenciji se uzima da mašina treba završiti rad tako da na kraju rada glava pokazuje na *prvi znak rezultata*. Stoga, u stanju q_2 mašina čita znakove, i sve dok su oni nule ili jedinice pomjera glavu ulijevo, i vraća se ponovo u stanje q_2 . Nailaskom na prvu prazninu, glava se pomiče udesno, i mašina prelazi u završno stanje q_4 . Ukoliko se u stanju q_1 umjesto nule nalazila jedinica, mašina je zamjenjuje sa nulom, i nastavlja dalje sa ciljem da “izvrši prenos”. Stoga mašina prelazi u stanje q_3 , nakon čega pomjera glavu jedno mjesto ulijevo i vraća se u stanje q_1 , tako da se proces nastavlja sa sljedećim bitom po težini. Proces se dalje nastavlja sve do pronalaska prve nule. Ukoliko se pri tome ne naiđe niti na jednu nulu (što je moguće ukoliko se zadani broj sastojao od samih jedinica), već samo na prazninu, na mjesto te praznine upisuje se jedinica, nakon čega mašina prelazi u završno stanje q_4 .

Ilustrirajmo opisani proces na jednom primjeru. Neka je kao ulazni podatak zadan binarni broj 10111. Računanje će se tada odvijati kroz sljedeći niz konfiguracija, pri čemu je radi preglednosti sekvenca koja predstavlja traku prikazana kao obični *string simbola*, dok je pozicija glave određena *podvučenim simbolom*:

$q_0: \underline{1}0111,$	$q_0: 1\underline{0}111,$	$q_0: 10\underline{1}11,$	$q_0: 101\underline{1}1,$	$q_0: 1011\underline{1},$	$q_0: 10111\underline{,}$
$q_1: 1011\underline{1},$	$q_3: 1011\underline{0},$	$q_1: 101\underline{1}0,$	$q_3: 101\underline{0}0,$	$q_1: 10\underline{1}00,$	$q_3: 10\underline{0}00,$
$q_1: 1\underline{0}000,$	$q_2: 1\underline{1}000,$	$q_2: \underline{1}1000,$	$q_2: \underline{1}1000,$	$q_4: \underline{1}1000$	

Prema izvornoj ideji, bilo je zamišljeno da same instrukcije Turingove mašine budu pohranjene u njenom "hardveru", odnosno da različitim tablicama prelaza Δ odgovaraju različite Turingove mašine (odnosno, svaka Turingova mašina je *preprogramirana* samo za određeni zadatak). Međutim, ubrzo je pokazano da je moguće konstruisati Turingovu mašinu (preciznije, mašinu sa takvom tablicom prelaza) koja bi bila u stanju da simulira *svaku drugu Turingovu mašinu* (uključujući i *samu sebe*) u smislu da bi se na traku kao ulazni podatak pored samih podataka koji treba da se obrađuju pohranila i tablica prelaza Turingove mašine koju treba simulirati (kodirana na odgovarajući način, u skladu sa upotrijebljenim alfabetom). Takva Turingova mašina naziva se **univerzalna Turingova mašina**. Slikovito rečeno, takva mašina je neka vrsta primitivne *programabilne računске mašine*, pri čemu se "program" (tj. opis Turingove mašine koju univerzalna Turingova mašina treba da simulira) nalazi pohranjen na samoj traci, zajedno sa podacima koji se obrađuju. Začudo, univerzalna Turingova mašina nije čak ni osobito komplicirana, pri čemu pod mjerom kompliciranosti smatramo broj instrukcija u njenoj tablici prelaza (nekoliko desetina instrukcija je sasvim dovoljno). Naravno, što se tiče same univerzalne Turingove mašine, njene vlastite instrukcije bile bi pohranjene u "hardveru". U nekom smislu, smatra se da otkriće univerzalne Turingove mašine predstavlja otkriće "softvera" (ne zaboravimo da su svi ovi pojmovi otkriveni *mnogo prije pojave stvarnih računara*, jer je model Turingove mašine predložen 1931. godine, dok su se prvi stvarni računari pojavili krajem drugog svjetskog rata, odnosno petnaestak godina kasnije). Drugim riječima, smatra se da je otkrićem univerzalne Turingove mašine prvi put pokazano da je *moguće nešto što danas nazivamo softverom*.

Što se tiče alfabeta Turingove mašine, pokazano je da se, za potrebe teorijskih razmatranja, možemo ograničiti na alfabet $\Gamma = \{\square, |\}$ koji se, pored praznine, sastoji od samo jednog simbola " $|$ ". Naime, pokazano je da je, bez obzira na korišteni alfabet, uvijek moguće konstruisati ekvivalentnu Turingovu mašinu koja bi koristila samo alfabet $\Gamma = \{\square, |\}$ i koja bi obrađivala podatke zapisane u **unarnoj notaciji**. Pri tome se pod unarnim zapisom broja n smatra string koji se sastoji od n istih simbola (obično se uzima simbol " $|$ "). Recimo, unarni zapis broja 7 glasi " $|||||||$ ".